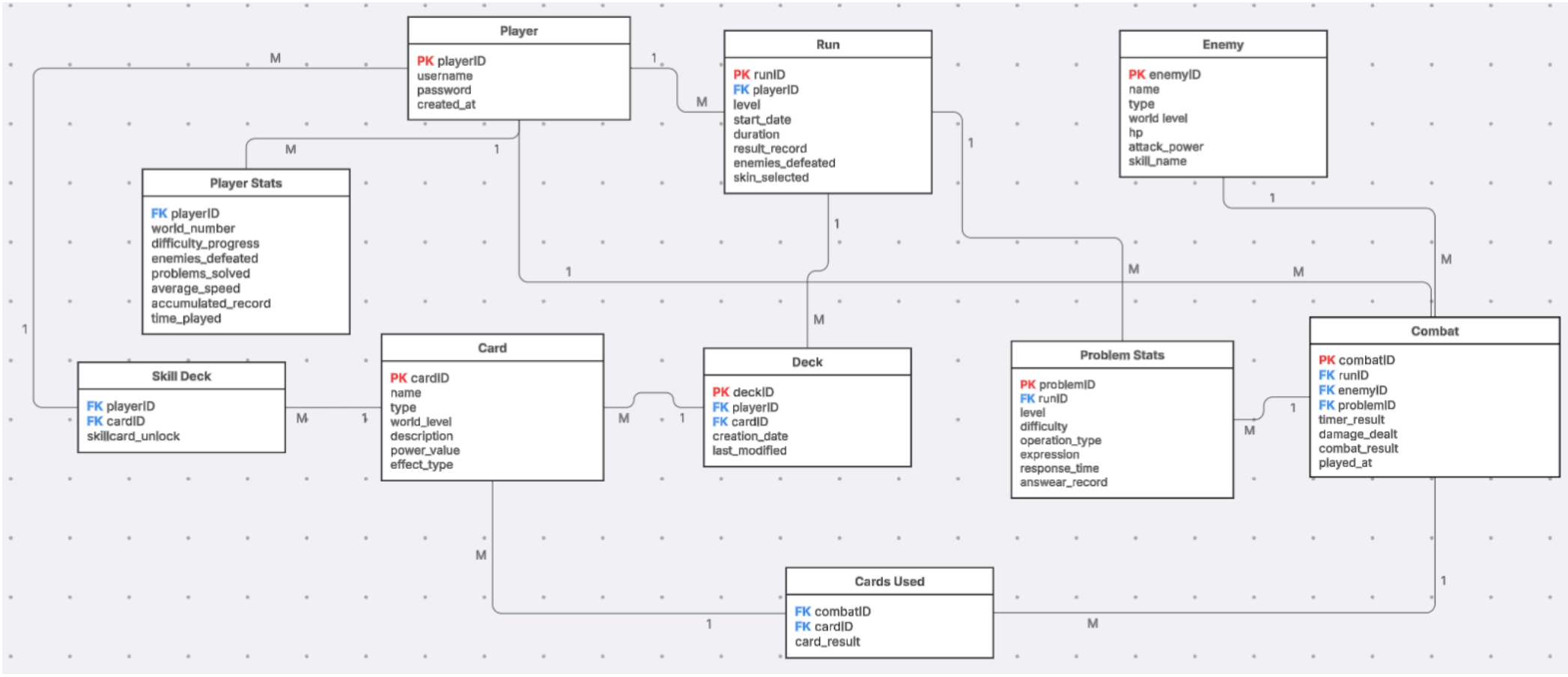
Player
PK playerID
username
password
created_at
Run
PK runID
FK playerID
level
start_date
duration
result_record
enemies_defeated
skin_selected
Enemy
PK enemyID
name
type
world level
hp
attack_power
skill_name
Player Stats
FK playerID
world_number
difficulty_progress
enemies_defeated
problems_solved
average_speed
accumulated_record
time_played
Skill Deck
FK playerID
FK cardID
skillcard_unlock
Card
PK cardID
name
type
world_level
description
power_value
effect_type
Deck
PK deckID
FK playerID
FK cardID
creation_date
last_modified
Problem Stats
PK problemID
FK runID
level
difficulty
operation_type
expression
response_time
answear_record
Combat
PK combatID
FK runID
FK enemyID
FK problemID
timer_result
damage_dealt
combat_result
played_at
Cards Used
FK combatID
FK cardID
card_result

Relationships, Cardinalities, and Integrity Constraints

Player - Run
Relationship: Player starts Run
Cardinality:
One player can have many runs.
Each run belongs to only one player.
Integrity constraints:
The Run table has playerID as a foreign key, this means a run cannot exist if there is no player connected to it. If a player is deleted, their runs are also deleted.

Player - Deck
Relationship: Player owns Deck
Cardinality:
One player can have one or more decks.
Each deck belongs to only one player.
Integrity constraints:
The Deck table has playerID as a foreign key, this makes sure every deck is connected to a real player. If the player is deleted, their deck records are deleted too.

Deck - Card
Relationship: Deck contains Card
Cardinality:
One deck can have many cards.
One card can be in many decks.
Integrity constraints:
This is a many-to-many relationship, so the table DeckCard is used. This table connects decks with cards. The primary key uses both deckID and cardID, so the same card cannot be repeated in the same deck. is_active shows if the card is currently being used in the active combat deck.

Player - SkillDeck
Relationship: Player unlocks Skill Card
Cardinality:
One player can unlock many skill cards.
Each skill deck record belongs to only one player.
Integrity constraints:
The SkillDeck table has playerID and cardID as foreign keys, this connects the skill card to both the player and the card table. There is also a unique constraint on (playerID, cardID), so the same player cannot unlock the same skill card more than once.

Card - SkillDeck
Relationship: Card is saved as Skill Card
Cardinality:
One card can appear in many skill deck records.
Each skill deck record uses only one card.
Integrity constraints:
The cardID in SkillDeck must exist in the Card table, this avoids writing the same card information again and keeps the database organized.

Run - ProblemStats
Relationship: Run generates ProblemStats
Cardinality:
One run can have many problem records.
Each problem record belongs to only one run.
Integrity constraints:
The ProblemStats table has runID as a foreign key, this means every math problem attempt must be connected to a real run. If a run is deleted, its problem records are also deleted.

Run - Combat
Relationship: Run contains Combat

Cardinality:
One run can have many combats.
Each combat belongs to only one run.
Integrity constraints:
The Combat table has runID as a foreign key, this makes sure every combat is part of a real run. If the run is deleted, the combat records are also deleted.

Enemy - Combat
Relationship: Enemy appears in Combat
Cardinality:
One enemy can appear in many combats.
Each combat has only one enemy.
Integrity constraints:
The Combat table has enemyID as a foreign key, this means a combat cannot reference an enemy that does not exist in the Enemy table.

ProblemStats - Combat
Relationship: ProblemStats is used in Combat
Cardinality:
One problem record can be connected to one or more combats.
Each combat can reference one problem record, but this field can also be empty.
Integrity constraints:
The problemID in Combat references ProblemStats(problemID). It can be null because some combats may have more than one problem or may not be connected to only one problem. But when a problem is added, it must already exist in the ProblemStats table.

Combat - CardsUsed
Relationship: Combat uses Card
Cardinality:
One combat can have many cards used.
Each CardsUsed record belongs to only one combat.

Integrity constraints:
The CardsUsed table has combatID as a foreign key, this means every card used must be connected to a real combat. If the combat is deleted, its card use records are deleted too.

Card - CardsUsed
Relationship: Card is used in Combat
Cardinality:
One card can be used in many combats.
Each CardsUsed record references only one card.
Integrity constraints:
The CardsUsed table has cardID as a foreign key. The primary key uses combatID, cardID, and turn_number. This prevents the same card from being recorded twice in the same turn of the same combat.

Player - PlayerStats
Relationship: Player has PlayerStats
Cardinality:
One player can have many player stats records.
Each player stats record belongs to only one player.
Integrity constraints:
The PlayerStats table has playerID as a foreign key. It also has a unique constraint on (playerID, world_number). This means each player can only have one stats record per world.

Normal Form Justification

First, it satisfies the First Normal Form because each field stores only one value. For example, the Card table stores one card per row, and the Combat table stores one combat event per row.

Second, it satisfies Second Normal Form because the information in each table depends on the primary key. For example, in the Combat table, values like damage_dealt, timer_result, and combat_result depend on the combatID. This means the table is storing information that actually belongs to that combat.

Third, it satisfies Third Normal Form because the database avoids repeating information in different places. For example, enemy details are stored only in the Enemy table, not repeated in every combat.